

Orientação a objetos

A Orientação a Objetos (OO) é um dos paradigmas mais importantes no desenvolvimento de software moderno. Ela se baseia em conceitos fundamentais como abstração, encapsulamento, herança e polimorfismo, que permitem organizar e simplificar sistemas complexos. Diferente dos paradigmas anteriores, como o imperativo, a OO facilita a manutenção, o reuso de código e a adaptação a novas funcionalidades. Ao longo da nossa aula, exploraremos esses conceitos detalhadamente, com exemplos práticos, para entender como eles transformaram a forma de desenvolver software e moldaram a evolução das linguagens de programação.

Abstração

Primeiramente, abordaremos o conceito de abstração, um dos pilares fundamentais desse paradigma de programação. A abstração nos permite gerenciar a complexidade dos sistemas ao focar nas características essenciais de um objeto, ignorando detalhes irrelevantes para o contexto atual. O conceito de classe e objeto será explicado em conjunto com exemplos práticos que facilitam o entendimento de sua aplicação. É importante ressaltar que essa aula também inclui, em tópicos futuros, os conceitos de encapsulamento, herança e polimorfismo, todos relacionados ao objetivo de organizar e simplificar o desenvolvimento de software.

Historicamente, a orientação a objetos surge como uma solução ao crescente problema da complexidade no desenvolvimento de programas, especialmente em relação ao reuso de código. No paradigma de programação estruturada, quanto maior o sistema, mais difícil se tornava mantê-lo e adaptá-lo a novas funcionalidades. Nesse cenário, os programas eram específicos e, frequentemente, não permitiam o reaproveitamento de rotinas. Com o advento da orientação a objetos, essa dificuldade foi mitigada, o que permitiu a criação de sistemas mais organizados, reutilizáveis e fáceis

de manter. A abstração é, portanto, uma ferramenta que permite selecionar apenas as características essenciais de um objeto, de acordo com o contexto do sistema em que ele será inserido.

Vamos pensar em um exemplo prático para ilustrar o conceito de abstração. Suponha que estamos desenvolvendo um sistema para gerenciar uma frota de automóveis. Um carro pode ser representado como um objeto no nosso sistema, e suas características mais relevantes podem variar de acordo com o contexto. Em um sistema de estacionamento, a placa do veículo é essencial, enquanto que em uma oficina, informações como a quilometragem ou o estado de conservação podem ser mais relevantes. Nesse caso, abstraímos apenas as informações pertinentes ao contexto de uso do carro, como a marca, o modelo, a cor, o ano e o número de portas. Além disso, podemos definir o comportamento do carro, como ligar, desligar, acelerar ou frear.

A classe “Carro”, nesse exemplo, encapsula os atributos (marca, modelo, cor, ano, número de portas) e os métodos (ligar, desligar, acelerar, frear). A partir dessa classe, podemos criar objetos específicos, como um Chevrolet Onix branco, ano 2024, com quatro portas. Esse objeto é uma instância da classe “Carro” e compartilha o comportamento definido na classe. Esse exemplo deixa claro que a abstração nos permite criar representações simplificadas dos objetos do mundo real, sem a necessidade de capturar todos os detalhes.

Outro exemplo prático é a abstração no contexto de um sistema para monitoramento de animais silvestres. Imagine que temos um objeto chamado “Valente”, um lobo-guará de dois anos. Para o nosso sistema, as informações relevantes sobre esse animal podem ser o nome, a espécie, a idade e o sexo. O comportamento que nos interessa monitorar pode ser andar, comer, uivar e caçar. Novamente, utilizamos a abstração para nos concentrar apenas nas características e comportamentos necessários para o objetivo do sistema, ignorando outros aspectos que seriam irrelevantes.

Encapsulamento

Encapsulamento é um dos conceitos centrais da orientação a objetos, e sua principal função é agrupar dados (atributos) e comportamentos (métodos) dentro de uma única unidade chamada classe. Enquanto o conceito de abstração nos ajuda a identificar os atributos e comportamentos relevantes de um objeto em um determinado contexto, o encapsulamento garante que esses dados e métodos estejam protegidos e organizados dentro da classe. Isso significa que, ao encapsular informações, criamos uma “cápsula” onde apenas os elementos definidos podem ser acessados ou manipulados, mantendo a integridade e segurança dos dados.

Um dos aspectos mais importantes do encapsulamento é a capacidade de controlar o acesso aos dados e métodos da classe. Alguns atributos e métodos podem ser privados, ou seja, acessíveis apenas pela própria classe, enquanto outros podem ser públicos, disponibilizados para interação com outras classes através de interfaces. Esse controle de acesso permite que a implementação de uma classe possa mudar internamente sem afetar os objetos que interagem com ela, desde que a interface pública permaneça inalterada. Esse recurso facilita a manutenção e evolução do software, uma vez que mudanças internas em uma classe não comprometem o comportamento esperado pelas outras.

Para ilustrar o conceito, imagine novamente o exemplo de um carro. Suponha que uma classe externa queira ligar o motor e acelerar o carro, mas não precisa saber detalhes técnicos como a taxa de compressão do motor ou a quantidade exata de combustível no tanque. O encapsulamento permite que o carro execute essas ações sem expor como os sistemas internos, como o motor e o combustível, funcionam. A outra classe apenas solicita que o carro execute uma ação, como “ligar” ou “acelerar,” sem saber como esses processos são implementados.

Essa ideia de encapsulamento pode ser comparada a uma “caixa preta”, onde a pessoa que interage com a classe não precisa conhecer seus detalhes internos, apenas os serviços disponíveis por meio da interface. No contexto dos automóveis, a Tesla é um excelente exemplo. A empresa pode atualizar o software de seus carros sem modificar o hardware, fazendo com que o veículo tenha um desempenho melhorado, como uma aceleração mais rápida, apenas por meio de mudanças no software. Esse processo só é possível

porque o software atua de forma encapsulada, sem a necessidade de expor os detalhes internos do carro.

Portanto, o encapsulamento não só protege os dados e facilita a manutenção do software, como também promove o reuso de código e a modularidade, permitindo que classes interajam entre si de maneira clara e organizada, sem comprometer o funcionamento interno de cada uma. Isso reforça a abstração, pois nos concentramos apenas nas interfaces, o que amplia a flexibilidade e o potencial de melhorias no sistema.

Herança

Herança é um conceito fundamental dentro da orientação a objetos e tem como principal função permitir que uma classe herde atributos e métodos de outra classe, promovendo o reuso de código e facilitando a manutenção do software. Vamos partir do exemplo que usamos anteriormente sobre carros. Já definimos a classe “Carro” com atributos como marca, modelo, cor, ano e portas, além de métodos como ligar, desligar, acelerar e frear. Agora, imagine que o cliente deseja incluir motocicletas no sistema. Ao analisar a classe “Moto,” percebemos que ela possui muitos atributos semelhantes aos da classe “Carro,” como marca, modelo, cor e ano. No entanto, ela também possui características e comportamentos exclusivos, como o atributo estilo (trail, esportiva, etc.) e o método empinar, que não faz sentido para um carro.

Ao invés de copiar e colar o código da classe “Carro” para criar a classe “Moto,” aplicamos o conceito de herança para criar uma superclasse mais genérica, chamada “Veículo.” Essa superclasse contém todos os atributos e métodos comuns a carros e motos, como marca, modelo, cor e ano, e métodos como ligar e desligar. A partir dessa superclasse “Veículo,” podemos criar as subclasses “Carro” e “Moto.” Cada subclasse herda os atributos e métodos da superclasse, mas também pode adicionar suas próprias características específicas. Por exemplo, a classe “Carro” tem o atributo portas e o método acionar freio de mão, enquanto a classe “Moto” tem o atributo estilo e o método empinar.

A herança, portanto, promove a reutilização de código, permitindo que classes especializadas aproveitem o que já foi implementado em classes mais gerais. Isso evita a repetição de código, seguindo o princípio DRY (Don't Repeat Yourself), uma regra importante no desenvolvimento de software. Além disso, a herança também facilita a manutenção do código. Se houver a necessidade de modificar algo na superclasse "Veículo", tanto a classe "Carro" quanto a classe "Moto" automaticamente se beneficiam dessas alterações, sem a necessidade de modificar diretamente essas subclasses.

Um exemplo interessante de como a herança é aplicada na prática pode ser observado no desenvolvimento de software para carros. A Tesla, por exemplo, pode realizar atualizações no software de seus veículos para melhorar o desempenho, como alterar o comportamento do motor para aumentar a velocidade de aceleração, sem alterar o hardware. Isso é possível graças ao conceito de herança, onde as classes especializadas herdam as funcionalidades da superclasse, permitindo melhorias contínuas sem impactar negativamente o funcionamento geral do sistema.

Em resumo, a herança promove a reutilização de código ao permitir que classes compartilhem atributos e métodos comuns, facilita a manutenção do software, pois as alterações em uma superclasse se propagam para as subclasses, e aumenta a extensibilidade, já que novas demandas podem ser atendidas sem comprometer o funcionamento existente.

Polimorfismo

Polimorfismo é um dos conceitos essenciais da orientação a objetos e trabalha em conjunto com os princípios de abstração, encapsulamento e herança. O termo "polimorfismo" vem do grego e significa "muitas formas," e sua aplicação em programação permite que objetos de classes diferentes respondam de maneiras distintas ao mesmo método. Para entender isso melhor, vamos voltar ao exemplo das classes "Carro" e "Moto," que herdamos da superclasse "Veículo." Sabemos que ambas as classes compartilham métodos como ligar, desligar, acelerar e frear, já que esses comportamentos estão definidos na superclasse. No entanto, a forma como cada veículo executa o método ligar, por exemplo, pode variar significativamente.

Em um carro, para ligá-lo, geralmente inserimos a chave, giramos uma vez para realizar verificações internas e depois mais uma vez para acionar o motor. Nos modelos mais recentes, podemos ter a opção de simplesmente apertar um botão com a chave presencial. Já em uma moto, embora o processo inicial seja semelhante (inserimos a chave e giramos para iniciar as verificações), para ligar o motor, apertamos um botão específico. Em motos mais antigas, sem partida elétrica, o piloto precisava usar a força física para acionar o motor. Assim, mesmo que o método “ligar” esteja presente tanto na classe “Carro” quanto na classe “Moto,” a implementação do comportamento é diferente em cada uma delas.

Esse é o ponto central do polimorfismo: o método “ligar” existe para ambos os veículos, mas é implementado de formas distintas. No código, isso se reflete em que cada classe filha, no caso “Carro” e “Moto,” implementa o método “ligar” à sua maneira, refletindo as particularidades de cada veículo. Com isso, o comportamento de um objeto pode variar conforme sua classe, mesmo quando o método é o mesmo. No futuro, caso decidíssemos adicionar classes como “Bicicleta Elétrica” ou “Trator,” esses veículos também teriam o método “ligar,” mas cada um implementaria essa funcionalidade de acordo com suas próprias especificidades.

Além de permitir que diferentes classes implementem o mesmo método de maneiras distintas, o polimorfismo também facilita a manutenção e a extensibilidade do código. Por exemplo, podemos manter um método genérico na superclasse que funcione bem para a maioria das subclasses, mas, se necessário, uma subclasse pode sobrescrever esse método e implementar um comportamento próprio. Isso mantém o sistema organizado e flexível, permitindo que novos tipos de objetos sejam facilmente integrados ao sistema sem a necessidade de grandes mudanças na estrutura existente.

Evolução das linguagens de programação

Hoje, vamos explorar a evolução das linguagens de programação, observando como chegamos ao paradigma de orientação a objetos, que é o foco do nosso curso. Programação é a transformação dos requisitos do cliente em uma linguagem compreensível para a máquina. Para isso, usamos

linguagens de programação, que atuam como intermediárias entre a linguagem humana e a linguagem que o computador entende. Assim como os idiomas naturais, as linguagens de programação evoluíram ao longo do tempo. Novas surgiram, outras caíram em desuso, e algumas ainda são usadas em contextos específicos. É importante entender essa evolução para sabermos como e por que chegamos ao paradigma orientado a objetos, além de estarmos preparados para trabalhar com sistemas que utilizam paradigmas mais antigos.

Nos anos 60, surgiram linguagens como Cobol e Fortran, voltadas para negócios e cálculos científicos, respectivamente. Essas linguagens seguiam o paradigma imperativo, onde os dados eram globais e podiam ser acessados por qualquer parte do código, o que trazia problemas de manutenção. Qualquer alteração em um dado poderia impactar todo o sistema. Com o tempo, isso se mostrou uma limitação séria, já que o crescimento dos sistemas aumentava a complexidade. Na década de 70, surgiram novas linguagens, como Algol 68 e Pascal, que introduziram o conceito de variáveis locais, permitindo que certas variáveis fossem acessíveis apenas em partes específicas do código, o que melhorou a organização e a manutenção.

Na metade da década de 70, com linguagens como C e Modula-2, os programas passaram a ser organizados em arquivos separados, uma melhoria significativa em relação aos grandes blocos de código único. No entanto, a complexidade continuou a crescer à medida que novos tipos de dados eram introduzidos. Foi nesse contexto que a programação orientada a objetos surgiu, no final da década de 80, trazendo uma solução para muitos dos problemas anteriores. Pela primeira vez, foi possível criar e trabalhar com tipos de dados abstratos de forma eficiente, com base na organização em torno de classes e no uso da herança para facilitar a reutilização e a extensão de código.

A programação orientada a objetos trouxe uma mudança radical, permitindo que os programas fossem mais facilmente organizados, mantidos e reutilizados. Ela superou muitas das limitações dos paradigmas anteriores, principalmente no que diz respeito à criação de tipos de dados abstratos e à modularização do código. Esse paradigma se tornou predominante na indústria e continua a ser a base para a maioria das novas linguagens de

programação, como Java, C++, C#, e até linguagens contemporâneas como TypeScript, que continuam a usar conceitos fundamentais da orientação a objetos.

Entender essa evolução é essencial não apenas para compreender o estado atual da programação, mas também para lidar com sistemas legados que podem utilizar paradigmas anteriores. Muitos sistemas antigos, programados em linguagens como Cobol, ainda estão em funcionamento, especialmente em setores como o governo e grandes corporações. Conhecer os diferentes paradigmas e suas origens nos ajuda a escolher a melhor abordagem para cada problema, seja adotando a orientação a objetos ou considerando paradigmas que melhor atendam às necessidades específicas de um sistema.

Concluimos nossa aula sobre orientação a objetos explorando seus conceitos centrais: abstração, encapsulamento, herança e polimorfismo. Esses pilares fornecem a base para o desenvolvimento de sistemas mais organizados, modulares e reutilizáveis. A evolução das linguagens de programação, desde os paradigmas iniciais até o surgimento da orientação a objetos, mostra como essa abordagem tornou o desenvolvimento de software mais eficiente e flexível.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “PROGRAMANDO com Abstração, Herança e Polimorfismo”, que está disponível no canal DevMedia no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamentos da programação de computadores**. 3.ed. Pearson: 2012.

BRAGA, P. H. **Teste de software**. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). **Arquitetura de software**. Pearson: 2017.

GALLOTTI, G. M. A. **Qualidade de software**. Pearson: 2015.

MEDEIROS, E. **Desenvolvendo software com UML 2.0 definitivo**. Pearson: 2004.

MORAIS, I. S. de. (Org.). **Engenharia de software**. Pearson: 2017.

PFLEEGER, S. L. **Engenharia de software: teoria e prática**. 2.ed. Pearson: 2003.

SOMMERVILLE, I. **Engenharia de software**. 10.ed. Pearson: 2019.